

Berkeley Function Calling Leaderboard(BFCL): Tool Use에서 Agentic LLM 평가로

Shishir G. Patil Huanzhi Mao Fanjia Yan Charlie Cheng-Jie Ji
Vishnu Suresh Ion Stoica Joseph E. Gonzalez

ICML 2025 / PMLR 267
한국어 재구성 번역본

원문: <https://openreview.net/forum?id=2GmDdhBdDk> / PMLR:
<https://proceedings.mlr.press/v267/patil25a.html>

Abstract

Function calling, 또는 tool use는 LLM이 외부 function, API, 사용자 정의 tool을 호출하는 능력을 뜻하며 agentic LLM application의 핵심 기능이다. 그러나 function call이 언제 유효한지를 판단하기 어렵고, 현실적인 function과 query를 폭넓게 확보하기 어렵기 때문에 표준 benchmark가 부족했다. 이 논문은 Berkeley Function Calling Leaderboard(BFCL)를 제안한다. BFCL은 single-turn, crowd-sourced, multi-turn, agentic dataset을 포함하며, Python/Java/JavaScript/REST API/SQL 등 여러 환경에서 serial 및 parallel function call을 평가한다. 핵심 방법은 실제 function 실행 없이도 대규모 deterministic evaluation을 가능하게 하는 Abstract Syntax Tree(AST) substring matching이다. 다양한 model을 평가한 결과, 최신 LLM은 single-turn call에서는 강하지만 memory, dynamic decision-making, long-horizon reasoning, stateful multi-step agentic setting은 여전히 어려운 과제로 남아 있다.

Contents

1	Introduction	3
2	Related Work	3
3	Berkeley Function Calling Leaderboard	4
3.1	Single-turn Dataset	4

⁰이 한국어 TeX/PDF는 공개 PDF와 PMLR 메타데이터에서 재구성한 번역본이다. OpenReview 직접 PDF/API 접근은 이 환경에서 challenge/403으로 차단되었고, PMLR/GitHub raw PDF는 접근 가능했지만 원저자 TeX source 또는 supplementary source archive는 확인한 공개 경로에서 발견되지 않았다. 따라서 본 파일은 저자 제공 원본 TeX가 아니라 PDF-only reconstruction이다.

3.2	Crowd-sourced Dataset	5
3.3	Multi-turn Dataset	5
3.4	Agentic Dataset	5
4	Evaluation Methodology	6
4.1	AST Substring Matching	6
4.2	Execution Response Matching	6
4.3	State-Based 및 Response-Based Evaluation	6
4.4	Exact-Match Evaluation	6
5	Results and Analysis	7
5.1	Parallel Function Call Ability	7
5.2	Multi-turn Error Analysis	7
5.3	Data Contamination Measurement	8
5.4	Memory Management Category	8
6	Conclusion	8
A	System Prompt for Prompting Models	8
B	Data Augmentation on Function Documents	9
C	Single Turn Dataset Formal Definition	9
D	Multi-turn Dataset Formal Definition	9
E	Data Generation Pipeline Implementation Details	9
F	LLM Judge BFCL Error Analyzer Implementation Details	10
G	Function Calling Model Performance over Time	10
H	BFCL AST Substring Matching	10
I	Function Parameters Distribution	10
J	Format Instruction Prompt for Agentic Task	10

1 Introduction

LLM은 자연어 이해, coding, reasoning, multimodal task에서 빠르게 발전했지만, 실제 application에서는 내부 지식만으로 충분하지 않은 경우가 많다. Coding assistant, knowledge discovery system, booking/commerce assistant, database interface처럼 외부 system과 연결되는 사용 사례에서는 web search, database, file system, user-defined API 등 external tool을 호출해야 한다. 이런 능력이 흔히 *function calling* 또는 *tool use*라고 불린다.

기존 연구인 Gorilla와 Toolformer는 LLM이 tool을 사용하는 방법을 학습시키는 방향을 열었고, 이후 LLM에 external tool을 연결하려는 관심이 커졌다. 하지만 function calling 평가 자체는 여전히 어렵다. APIBench, ToolBench, ToolSandbox, Nexus Function Calling Leaderboard 등 기존 benchmark는 functional correctness만 보거나, 실제 user interaction을 충분히 반영하지 못하거나, LLM-based evaluator에 의존하여 reproducibility와 objectivity 문제가 생긴다. 특히 parallel invocation, missing parameter, irrelevant tool, multi-turn state update, long-context memory 같은 현실적 pattern이 부족하다.

BFCL은 이러한 공백을 메우기 위해 설계된 multi-task, multi-turn benchmark이다. 저자들은 BFCL이 다음 기여를 한다고 정리한다.

1. Python, Java, JavaScript, REST API, SQL을 포함하는 5,551개의 question-function-answer pair로 구성된 diverse dataset을 제공한다.
2. 실제 execution 없이도 scalable하고 deterministic하게 평가할 수 있도록 AST 기반 substring matching을 function execution의 proxy로 사용한다.
3. 2024년 2월 26일부터 4월 1일까지 수집한 real-world, user-contributed query를 포함하여 community-contributed dataset을 추가한다.
4. multi-turn 및 agentic dataset으로 memory, state, response path, abstention, dynamic decision-making을 평가한다.

2 Related Work

Function calling은 LLM을 외부 system과 연결하는 agentic behavior의 기반이다. Toolformer, Gorilla, ToolLLM, LLMCompiler 같은 연구는 tool selection, API invocation, parallel execution, external knowledge retrieval을 통해 LLM의 활용 범위를 확장했다. 그러나 benchmark 관점에서는 몇 가지 한계가 있었다.

APIBench는 function call의 executable correctness를 보지만 semantic variation과 API usage diversity가 제한된다. NexusRaven 및 관련 function calling leaderboard는 coverage가 좁거나 특정 model/API format에 치우칠 수 있다. ToolBench와 StableToolBench는 RapidAPI 또는 simulated API response에 의존하며, response solvability 판정에 LLM evaluator가 들어가 model-induced bias를 만들 수 있다. ToolSandbox와 TauBench는 user simulator를 LLM으로 구성하기 때문에 hallucina-

tion 또는 instruction-following error가 benchmark 결과에 섞일 수 있다. T-Eval 역시 LLM-based evaluation의 영향을 받는다.

BFCL은 deterministic metric, 더 넓은 function set, 실제 user-contributed query, multi-language query, Python/REST 외 Java/JavaScript/SQL coverage, multi-turn stateful interaction을 결합하여 이러한 한계를 직접 겨냥한다.

3 Berkeley Function Calling Leaderboard

BFCL은 네 범주의 dataset으로 구성된다. 그림 1의 원형 chart는 inner ring에 네 major section, middle ring에 evaluation method, outer ring에 category를 배치하여 benchmark의 범주를 보여준다. PDF-only reconstruction에서는 원본 그림을 재삽입하지 않고 해당 구조를 표로 재구성한다.

범주	핵심 평가	설명
Single-turn	AST / Execute	한 user prompt에서 하나 이상의 function call을 생성하는 능력. simple, multiple, parallel, parallel multiple, irrelevance/relevance를 포함한다.
Crowd-sourced	AST / Execute	실제 user query에서 출발한 function calling entry. 더 많은 function 수와 parameter 수를 포함하며 real-world distribution을 반영한다.
Multi-turn	AST / State / Response	여러 turn에서 user request가 진화할 때 missing parameter, missing function, long context, state update를 처리하는 능력을 평가한다.
Agentic	Exact match / stateful memory	Live relevance, memory management, SQL 등 agentic setting에서 abstain, answer field, stateful memory를 평가한다.

표 1: BFCL의 네 major section과 evaluation focus. 원문 Figure 1을 텍스트/표 형태로 재구성했다.

3.1 Single-turn Dataset

Single-turn function-calling scenario는 available tool 수와 invocation pattern에 따라 다섯 가지로 나뉜다. Simple은 하나의 tool과 하나의 invocation, Multiple은 여러 tool 중 각각 한 번의 invocation, Parallel은 하나의 tool을 여러 번 호출, Parallel Multiple은 여러 tool을 병렬로 여러 번 호출하는 경우이다. Irrelevance는 tool이 주어졌지만 호출하면 안 되는 경우를 뜻한다.

평가는 두 방식으로 수행된다. AST-based evaluation은 Python, Java, JavaScript function을 popular GitHub repository에서 수집하고 trivial function을 제거한 뒤, model output을 parse하여 function

name과 parameter value가 ground truth set에 들어가는지를 확인한다. Execute evaluation은 math/physics computation, REST API, SQL-like task처럼 실제 실행 가능한 backend를 사용한다.

3.2 Crowd-sourced Dataset

Crowd-sourced dataset은 2024년 2월 26일부터 4월 1일까지 실제 user query에서 수집된 entry를 기반으로 한다. 기존 single-turn benchmark가 synthetic 또는 expert-curated query에 치우칠 수 있다는 문제를 줄이기 위해 real user interaction을 도입했다. 이 범주는 single-turn보다 더 많은 function 수와 더 복잡한 parameter 구조를 포함한다. 예를 들어 crowd-sourced entry에는 최대 37개 function이 등장하고, 어떤 function은 21개 parameter를 갖는 반면, single-turn의 평균 최대치는 훨씬 낮다.

3.3 Multi-turn Dataset

Multi-turn dataset은 user와 assistant 간 전체 conversation을 평가한다. 하나의 conversation은 여러 turn으로 구성되고, 각 turn은 assistant와 tool/environment 간 하나 이상의 step을 포함할 수 있다. BFCL은 multi-turn을 Base, Missing Parameters, Missing Functions, Long Context로 나눈다.

Base category는 필요한 정보가 모두 들어 있는 일상 task를 다룬다. Missing Parameters는 critical parameter가 user request에 없고 system에서도 추론할 수 없을 때 model이 clarification 또는 error를 반환하는지 평가한다. Missing Functions는 주어진 function 중 request를 처리할 수 있는 function이 없음을 인식해야 하는 경우이다. Long Context는 긴 multi-turn query 또는 function call result 속에서도 accuracy를 유지하는지 평가한다.

Dataset 구축에는 vehicle control, trading bot, travel booking, file system, messaging, Twitter, ticket booking, math 등 여러 domain을 아우르는 custom API codebase가 사용된다. 각 domain은 API state와 design이 투명하게 정의되어 있어 state-based evaluation이 가능하다.

3.4 Agentic Dataset

Agentic dataset은 function call 자체를 넘어, real deployment에 가까운 stateful agent behavior를 평가한다. Live relevance category는 answer field에 대한 exact-match evaluation을 사용한다. Memory management category에서는 여러 domain-level dialogue block을 통해 user-specific fact가 점진적으로 드러나고, 각 block 이후 memory snapshot이 저장된다. 이후 empty chat history와 snapshot access만으로 query를 해결하게 하여 model이 정보를 retrieve, add, overwrite, delete할 수 있는지 본다.

SQL dataset은 각 question을 SQL query로 번역할 수 있는 setting을 다룬다. 전통적인 text-to-SQL처럼 raw SQL string을 생성하고 exact match로 평가하는 대신, BFCL은 SELECT, INSERT, UPDATE, DELETE 같은 fundamental SQL operation을 JSON schema로 제공하고 필요한 structured parameter를 채우게 한다.

4 Evaluation Methodology

4.1 AST Substring Matching

실제 function execution은 대규모 benchmark에서 비용과 reproducibility 문제가 크다. BFCL은 programming language literature에서 착안하여 model output을 AST로 parse하고, function name과 parameter value를 systematic하게 비교한다. Model output은 prompt instruction을 통해 Python-callable function call 형태로 제한된다. 이후 Python의 ast module을 사용해 function name과 parameter를 추출한다.

Exact parameter string match 대신 각 parameter가 predefined valid value set에 속하는지 확인한다. Function name이 정확히 일치하고 모든 parameter value가 가능한 정답 집합에 포함되면 call이 correct로 간주된다. 저자들은 subset에서 execution-based metric과 AST metric을 비교했고, Figure 3에서 두 metric이 강하게 correlated됨을 보였다. 따라서 AST matching은 actual execution의 scalable offline proxy로 사용될 수 있다.

4.2 Execution Response Matching

Execution response matching은 function을 실제 실행하고 expected outcome과 비교한다. Pure Python function, REST API, SQL-like operation처럼 실행 가능한 backend에서 사용된다. 이 방식은 functional correctness를 직접 측정하지만, 모든 benchmark entry에 적용하기 어렵기 때문에 AST matching과 보완적으로 사용된다.

4.3 State-Based 및 Response-Based Evaluation

Multi-turn/agent setting에서는 correct final answer만으로 충분하지 않다. State-Based Evaluation은 각 turn 이후 system final state를 ground truth state와 비교한다. 여러 function call sequence가 같은 state를 만들 수 있으므로, final state가 맞으면 correct로 볼 수 있다. 이는 file creation, stock watchlist update처럼 state-changing action에 적합하다.

Response-Based Evaluation은 read-only request에서 특히 중요하다. 예를 들어 weather query에서 model이 그냥 답을 추측하는 대신 get zipcode by city 다음 get weather by zipcode를 호출했는지를 확인해야 한다. 따라서 BFCL은 state-based와 response-based check를 결합해 decision-making process를 더 깊게 본다.

4.4 Exact-Match Evaluation

Agentic task 중 dedicated answer field를 요구하는 경우, BFCL은 system prompt의 formatting instruction에 따라 answer field만 strict exact match로 평가한다. 이렇게 하면 reference phrase가 긴 설명 안에 우연히 나타나서 false positive가 되는 문제를 줄일 수 있다.

5 Results and Analysis

원문 Table 1은 여러 LLM의 BFCL 성능을 single-turn, multi-turn, live relevance, memory management 등 category별로 비교한다. PDF text extraction에서 표 전체는 column wrapping 때문에 원형 복원이 어렵지만, 핵심 trend는 명확하다. GPT-4o 계열은 single-turn function call에서 높은 성능을 보이며, Qwen2.5-72B-Instruct 역시 prompt 기반 single-turn AST category에서 강하다. 그러나 overall accuracy는 memory, dynamic decision-making, long-horizon multi-turn reasoning이 포함되면서 낮아진다.

관찰	의미
Single-turn 성능은 최신 model에서 높다	Function format을 알고 있고 tool documentation이 충분하면 model은 simple/multiple/parallel call을 상당히 잘 생성한다.
Crowd-sourced single-turn 과 분 포 가 다르다	실제 user query는 multiple function 선택과 복잡한 parameter를 더 자주 요구한다.
Multi-turn error가 성능 병목이다	Empty response, state mismatch, execution response mismatch, force termination, API error 등이 긴 interaction에서 증가한다.
Memory management는 어렵다	Short-term/long-term fact를 snapshot에서 retrieve/add/overwrite/delete하는 능력은 아직 안정적이지 않다.
Data contamination 분석이 필요하다	공개 single-turn dataset에 대한 familiarity와 새 crowd-sourced dataset에 대한 familiarity 차이를 perplexity/CharNLL로 비교한다.

표 2: BFCL 결과의 practitioner-oriented 요약. 원문 Table 1-2와 Figure 6-8의 핵심 trend를 재구성했다.

5.1 Parallel Function Call Ability

Crowd-sourced dataset에서는 multiple scenario가 많고 parallel scenario는 상대적으로 적다. 이는 실제 user가 한 turn에서 같은 function을 대량 병렬 호출하기보다, 여러 function 중 어떤 것을 선택할지 요구하는 경우가 많다는 점을 시사한다. Model 평가에서도 parallel function call은 format과 parameter alignment가 맞아야 하므로 error가 발생하기 쉽다.

5.2 Multi-turn Error Analysis

BFCL은 deterministic analysis와 LLM-as-a-Judge analysis를 함께 사용해 multi-turn failure를 분류한다. Deterministic category는 Empty Turn Response Error, Instance State Mismatch, Execution Response Mismatch, Force Termination, API Error를 포함한다. Judge 기반 분석은 model이 function documentation을 오해했는지, user intent를 잘못 해석했는지, exploration step과 critical failure step을 구분했는지 더 설명적으로 분류한다.

5.3 Data Contamination Measurement

저자들은 single-turn dataset이 공개되어 model training data에 노출되었을 가능성을 검토하기 위해 perplexity와 character-level negative log likelihood(CharNLL)를 비교한다. Crowd-sourced dataset은 single-turn보다 나중에 수집되었기 때문에 familiarity 비교의 대조군 역할을 한다. 원문 Table 2는 Llama2-7B, OpenFunctions-v2, CodeLlama-7B, Meta-Llama-3 8B, Mistral-7B, Functionary-7B 등이 single-turn dataset에서 crowd-sourced보다 낮은 perplexity를 보이는 경향을 제시한다. 이는 일부 model이 공개 benchmark distribution에 더 familiar할 가능성을 시사하지만, 그 자체가 곧 memorization 증거는 아니다.

5.4 Memory Management Category

Memory benchmark는 topic boundary와 temporal gap을 포함한다. Model은 empty chat history 상태에서 memory snapshot을 사용하여 이전에 언급된 user-specific fact를 찾아야 한다. 이는 production assistant에서 persistence와 personalization이 중요한 만큼, 단순 function call benchmark보다 더 현실적인 agentic requirement를 반영한다.

6 Conclusion

BFCL은 function calling 평가를 single-turn API invocation에서 agentic LLM evaluation으로 확장한다. 핵심은 deterministic하고 scalable한 AST matching, real user query를 반영한 crowd-sourced dataset, multi-turn state/response evaluation, memory 및 SQL category를 결합한 점이다. 결과적으로 최신 model은 simple single-turn tool use에서는 강하지만, real-world deployment에 필요한 memory, dynamic decision-making, long-horizon reasoning, stateful interaction에서는 여전히 큰 개선 여지가 있음을 보여준다. BFCL은 function-calling model과 agent framework가 앞으로 무엇을 개선해야 하는지 측정하는 실용적 기준점이 된다.

Impact Statement

BFCL은 LLM의 function-calling capability를 더 체계적으로 평가함으로써, external system과 연결되는 AI application의 reliability, safety, reproducibility를 높이는 데 기여한다. 다만 benchmark가 leaderboard로 널리 쓰일수록 model이 benchmark format에 overfit하거나, 공개 dataset contamination 문제가 발생할 수 있다. 따라서 저자들이 제안한 crowd-sourced update, data contamination check, deterministic evaluation은 benchmark 운영에서 중요한 장치다.

A System Prompt for Prompting Models

Appendix A는 prompting model이 function-calling action을 수행하도록 하는 universal system prompt를 제시한다. Prompt는 model에게 question과 possible functions가 주어졌을 때 하나 이상의 function/

tool call을 구성하라고 지시한다. 사용할 function이 없으면 그 점을 지적하고, required parameter가 부족하면 clarification 또는 error를 반환해야 한다. Function을 호출하기로 했다면 response를 function call format으로만 반환하도록 제한한다. 이 appendix는 prompt-only model과 native function-calling model을 같은 benchmark format 안에서 비교하기 위한 bridge 역할을 한다.

B Data Augmentation on Function Documents

Appendix B는 function document와 user query를 변형하는 data augmentation 절차를 설명한다. Multiple Parallel Functions Augmentation은 query와 function set을 결합해 model이 여러 function call을 처리하고 unused function을 filtering하는 능력을 테스트한다. Function Parameter Removal은 prompt에서 parameter 일부를 제거하여, model이 부족한 정보를 추측해서 call하지 않고 follow-up question 또는 error를 내는지 본다.

C Single Turn Dataset Formal Definition

Appendix C는 single-turn category를 수식으로 정의한다. Function set을 F , predicted call set을 $\hat{f}(P)$ 라고 할 때 Simple은 $|F| = 1$ 이고 $|\hat{f}(P)| = 1$ 인 경우, Multiple은 $|F| > 1$ 이고 단일 invocation인 경우, Parallel은 single tool이 여러 번 호출되는 경우, Parallel Multiple은 multiple tool과 multiple invocation이 동시에 등장하는 경우이다. Relevance는 적어도 하나의 function call이 기대되는 경우이고, Irrelevance는 tool이 제공되어도 호출하지 않아야 하는 경우이다.

D Multi-turn Dataset Formal Definition

Appendix D는 multi-turn conversation, turn, step, environment state를 formal하게 정의한다. 핵심은 user message가 여러 turn에 걸쳐 들어오고, assistant가 각 turn에서 tool/environment와 여러 step으로 상호작용하며, evaluation은 state와 response trajectory를 모두 본다는 점이다. Missing parameter, missing function, long context는 이 formalism에서 각기 다른 failure mode를 유도한다.

E Data Generation Pipeline Implementation Details

Appendix E는 single-turn, crowd-sourced, multi-turn dataset 생성 pipeline을 설명한다. Single-turn에서는 GitHub repository function을 수집하고 trivial function을 제거하며, executable Python/REST/SQL task를 별도로 구성한다. Crowd-sourced dataset은 real user query를 preprocessing하고, minimum edit principle로 clarity와 consistency를 높인다. Multi-turn dataset은 vehicle control, trading bot, travel booking, file system, messaging, Twitter, ticket booking, math 등 domain-specific API codebase에서 task를 생성하고, API state와 expected outcome을 검증한다.

F LLM Judge BFCL Error Analyzer Implementation Details

Appendix F는 GPT-4o 기반 judge를 사용해 multi-turn failure root cause를 분류하는 prompt를 제시한다. Judge는 final state가 ground truth와 다른지, minimum required trajectory를 따르지 않았는지, exploration step과 critical failure step을 구분해야 한다. Failure가 없으면 정확히 하나의 “No Failure” entry를 반환하게 한다. 이 절차는 deterministic checker가 알려주는 failure signal에 설명 가능성을 더한다.

G Function Calling Model Performance over Time

Appendix G와 Figure 8은 2023년과 2024년 초 model이 대규모 function calling에서 어려움을 겪었지만, model size 증가와 function calling post-training objective 도입 이후 성능이 개선되었음을 시각화한다. 이는 BFCL이 단일 snapshot benchmark가 아니라 시간이 지남에 따라 model capability 변화를 추적하는 leaderboard로 기능한다는 점을 보여준다.

H BFCL AST Substring Matching

Appendix H는 AST substring matching 절차를 자세히 설명한다. Model response를 parse하여 function information을 systematic하게 추출하고, parameter type과 value를 비교한다. Integer와 float handling은 language별 차이가 있다. Python에서는 int가 float 자리에서 자동 변환될 수 있지만, Java/JavaScript에서는 documentation이 float를 요구하면 5가 아니라 5.0 같은 literal float가 필요하다. List와 tuple은 order가 중요하며, order-agnostic question은 correct answer의 permutation을 모두 열거한다. Dictionary list에서는 list order는 중요하지만 dictionary 내부 key order는 중요하지 않다. Java/JavaScript의 code construct string은 Tree-sitter로 Python equivalent로 변환한 뒤 평가한다.

I Function Parameters Distribution

Appendix I와 Figure 10은 single-turn과 crowd-sourced category의 function 수와 parameter 수 분포를 비교한다. Crowd-sourced entry는 function 수와 parameter 수의 range와 mean이 모두 더 크다. 이는 real-world user query가 synthetic single-turn보다 훨씬 복잡한 tool selection과 argument construction을 요구한다는 근거다.

J Format Instruction Prompt for Agentic Task

Appendix J는 agentic dataset에서 model이 최종 answer를 반환할 때 요구되는 추가 system prompt를 제시한다. Model은 final answer를 answer와 context field가 있는 형식으로 반환해야 하며, 모르면 두 field 모두 “I do not know”로, 질문에 답할 수 없으면 “I cannot answer this question”으로 응답해야

한다. 이 format restriction은 exact-match evaluation을 가능하게 하고, 장황한 설명 속 우연한 phrase match를 줄인다.

References 요약

원문 references는 Toolformer, Gorilla/APIBench, ToolBench, ToolSandbox, TauBench, NexusRaven, LLMCompiler, FreshLLMs, AppWorld, xLAM, GPT-4, Llama 3, Text-to-SQL benchmark 등 function calling, tool-use, agent evaluation, external knowledge augmentation, SQL evaluation 관련 연구를 폭넓게 인용한다. 자세한 bibliographic record는 원문 PDF의 References section을 참조한다.